Vitis AI Installation and Setup

This document provides a step-by-step guide to install and configure Vitis AI on a Windows development machine using WSL and Docker. The tutorial covers the required software components, the basic setup procedure, and simple verification steps to ensure that the Vitis AI environment is working correctly.

In this tutorial, you will learn how to:

- Install and enable Windows Subsystem for Linux (WSL)
- Install Ubuntu 22.04 LTS inside WSL
- Install and configure Docker Desktop
- Download and run a Vitis AI Docker container
- Launch a Jupyter Notebook environment for running deep learning tutorials

** Vitis AI 3.5 official guide: Vitis AI User Guide (UG1414)

1. Install Windows Subsystem for Linux (WSL) and Ubuntu 22.04 LTS

- Official guide: Install Ubuntu on WSL 2
- Requirement: Windows 11 (recommended) or Windows 10 version 21H2 or later on a physical machine (not inside another virtual machine)

1.1 Install WSL

To install Ubuntu, you first need to install and enable WSL on your Windows machine.

From the Windows search bar, open a PowerShell terminal and run:

```
#PowerShell
wsl --install
```

You may be prompted to grant admin permission to continue the installation. Depending on your system, you may also need to reboot your machine before installing and running any Ubuntu distro. On some systems, the `wsl --install` command may automatically install the latest available Ubuntu version by default. For the Docker-based Vitis AI workflow, the recommended Ubuntu version is 22.04 LTS.

1.2 Install Ubuntu 22.04 LTS (recommended version)

In a PowerShell terminal, to see a list of all available distros and versions, run:

```
#PowerShell
wsl --list --online
```

To install the Ubuntu 22.04 LTS distribution, run:

```
#PowerShell
wsl --install Ubuntu-22.04
```

During installation of an Ubuntu distro on WSL, you are asked to create a username and password specific to that instance. This also starts an Ubuntu session and logs you in.

1.3 Enable Ubuntu 22.04 LTS in WSL

After installation, you can start an Ubuntu instance from PowerShell using: `wsl ~ -d <Distro>`. In this case:

```
#PowerShell
wsl ~ -d Ubuntu-22.04
```

where the symbol ~ starts an instance from PowerShell with the Ubuntu home as the working directory.

Alternatively, you can use the **WSL terminal application directly**, if Ubuntu-22.04 is set as the default distribution. In this case, you can simply open the WSL application from the Windows Start menu, and it will automatically launch the default Linux distribution. To set Ubuntu-22.04 as the default distribution, run:

```
#PowerShell
wsl --set-default Ubuntu-22.04
```

2. Install Docker Desktop on Windows

Docker is a containerization platform that allows applications to run inside isolated environments called *containers*. Docker-based installation is the recommended approach for most users as it provides a pre-configured environment with all dependencies (Python, frameworks, libraries, compilers) already installed. Containers are isolated from the host operating system. This isolation prevents dependency conflicts and ensures that the Vitis AI environment does not interfere with the host system configuration, ensuring reproducibility across different machines.

2.1 Install Docker Desktop

To use the Docker-based Vitis AI flow on Windows, you must install Docker Desktop.

1. Download Docker Desktop for Windows from the official website

Specifically download: **Docker Desktop for Windows – x86_64.exe**

2. Run the executable file and follow the default installation instructions.

3. After completing the installation, open Docker Desktop and enable WSL integration.

    - Go to **Docker Desktop → Settings → Resources → WSL Integration**
    - Enable: ☑ Enable integration with my default WSL distro
    - In the list below, select the specific distribution you want to use (e.g., Ubuntu-22.04)
    - Click "Apply & restart"

4. Reboot your computer.

2.2 Verify Docker Installation and WSL Integration

After restarting your computer:

- Reopen the **Docker Desktop** application
- Open your Ubuntu WSL terminal, following the commands described in section **"1.3 Enable Ubuntu 22.04 LTS in WSL"**. Again, you can do this from a PowerShell terminal by running:

```
#PowerShell
wsl ~ -d Ubuntu-22.04
```

Alternatively, you can open the WSL terminal application directly if you have set Ubuntu-22.04 as the default distribution.

- Now perform a quick verification of your Docker installation by executing:

```
# Ubuntu (WSL)
docker run hello-world
```

This command downloads a test image from Docker Hub and runs it inside a container. If the installation is correct, the container will print a **"Hello from Docker!"** message.

2.3 (Optional) In case of permission issues.

In case of issues with the `hello-world` test, follow the Linux post-installation steps to ensure that your user belongs to the `docker` group.

After opening your Ubuntu distribution in WSL, add your user to the `docker` group:

```
# Ubuntu (WSL)
sudo usermod -aG docker $USER
exit
```

Then reopen Ubuntu in WSL and run the following command to activate the updated group membership:

```
# Ubuntu (WSL)
newgrp docker
```

Finally, reboot your computer to ensure all changes are fully applied.

After restarting your system, reopen the Docker Desktop application and then open your Ubuntu WSL distribution. Verify the configuration again using the hello-world test:

```
# Ubuntu (WSL)
docker run hello-world
```

If the command executes successfully and prints the Docker welcome message, the configuration is correct.

3. Install Vitis AI 3.5

- Official guide: Vitis AI 3.5 Host Installation Instructions

The Vitis-AI Docker containers come in three main variants, supporting different hardware platforms and deep learning frameworks:

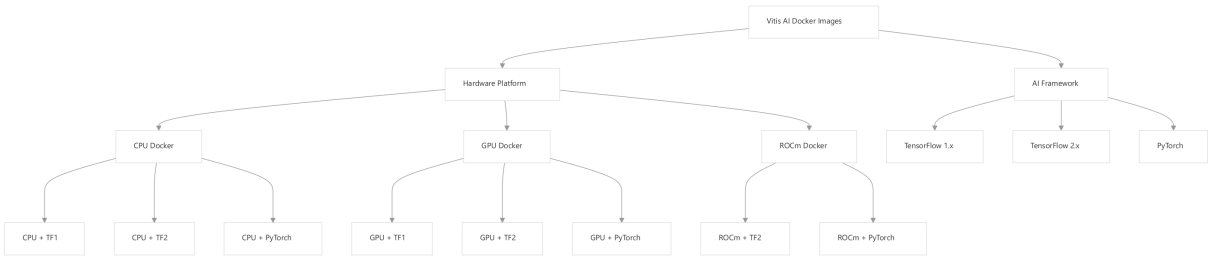

Figure 1 – Vitis AI Docker Image Types: DeepWiki Xilinx/Vitis-AI

- **CPU-only**: For development and testing without GPU acceleration
- **GPU-enabled**: For accelerated training and quantization with NVIDIA GPUs
- **ROCm-enabled**: For AMD GPU acceleration support

Each container variant is available with different framework configurations:

- TensorFlow 1.x
- TensorFlow 2.x
- PyTorch

The Docker images follow a consistent naming convention:

`xilinx/vitis-ai-<framework>-<hardware_platform>:<version>`

Examples:

`xilinx/vitis-ai-tensorflow2-cpu:latest` -- TensorFlow 2.x on CPU docker

`xilinx/vitis-ai-tensorflow2-gpu:latest` -- TensorFlow 2.x with NVIDIA GPU support

`xilinx/vitis-ai-pytorch-rocm:latest` -- PyTorch with AMD GPU/ROCm support

There are two main ways to obtain a Vitis AI container:

- **Use a pre-built Docker container** available on Docker Hub
- Build a custom Docker container locally using the scripts provided in the Vitis AI repository

Pre-built containers are only available for CPU and ROCm environments. If you want to use CUDA-enabled NVIDIA GPUs, the Docker container must be built locally from the Vitis AI repository scripts

In this tutorial, we will use the **CPU-only pre-built** container, which can be downloaded directly from Docker Hub. Specifically, we will use the **Vitis AI 3.5 TensorFlow 2.x container**, which includes the TensorFlow 2 framework and the Vitis AI development tools: `xilinx/vitis-ai-tensorflow2-cpu:latest`.

To install it, follow the instructions below.

3.1 Obtain Vitis AI 3.5.

Make sure that the Docker Desktop application is running and that you are inside your Ubuntu WSL distribution (again, you can follow the commands described in section **"1.3 Enable Ubuntu 22.04 LTS in WSL"**).

Clone the Vitis AI GitHub repository. For this tutorial, we will specifically use Vitis AI 3.5:

```
#Ubuntu (WSL)
git clone --branch v3.5 https://github.com/Xilinx/Vitis-AI.git
cd Vitis-AI
```

3.2 Leverage the Pre-Built Docker

First, download the Vitis AI TensorFlow 2 CPU container from Docker Hub:

```
#Ubuntu (WSL)
docker pull xilinx/vitis-ai-tensorflow2-cpu:latest
```

Next, start the container using the provided script:

```
#Ubuntu (WSL)
./docker_run.sh xilinx/vitis-ai-tensorflow2-cpu:latest
```

The script `docker_run.sh` launches the container with the correct configuration for the selected hardware platform and mounts the required directories. If everything worked correctly, you should see a terminal inside the Vitis AI Docker container! **Congratulations**!

Inside the container, activate the TensorFlow 2 environment:

```
#Ubuntu (WSL)
conda activate vitis-ai-tensorflow2
```

to verify that it works properly.

At this point, the Vitis AI environment is correctly configured. Exit the container because in the next step we will start it again together with a **Jupyter Notebook server**.

```
#Ubuntu (WSL)
exit
```

4. Run the Container with Jupyter Notebook

We are ready to start the tutorial on **training and quantization** of a simple neural network. In this exercise, we will use the **MNIST dataset**, a well-known benchmark dataset in machine learning that contains thousands of handwritten digit images (0–9) commonly used for training and evaluating classification models.

Download the file `mnist_vitisai_tf2.zip` from Canvas, extract it, and place the folder on your **Desktop**. This folder contains two Jupyter notebooks, which will be used in the next tutorial to train and quantize a neural network:

- `training.ipynb`
- `quantization.ipynb`

Now start the container again while mounting the notebook directory as Vitis AI working directory and enabling Jupyter access:

```
#Ubuntu (WSL)
docker run -it --rm \
  -v /mnt/c/Users/<YOUR_WINDOWS_USERNAME>/Desktop/mnist_vitisai_tf2:/workspace \
  -p 8080:8080 \
  xilinx/vitis-ai-tensorflow2-cpu:latest \
  bash
```

Note that the command `-v host_path:/workspace` mounts your local project folder inside the container. Replace `<YOUR_WINDOWS_USERNAME>` with your actual Windows username. Note that if you saved the folder in a different location, you must modify the host path accordingly to match the actual location of your project directory.

Activate the Vitis AI TensorFlow environment:

```
#Ubuntu (WSL)
conda activate vitis-ai-tensorflow2
```

Start the Jupyter Notebook server:

```
#Ubuntu (WSL)
jupyter notebook --ip=0.0.0.0 --port=8080
```

If everything worked correctly, the terminal will display a link similar to the following:

`http://127.0.0.1:8080/?token=...`

Open this link in your browser to access the Jupyter Notebook interface, where you can run the provided notebooks. **Enjoy!**